

The Central Processing Unit (CPU) and
Machine Language

**CSCI 1101 Introduction to
Computer Science**

Computer System

Computer System

Hardware

All those parts of a computer that you can see and touch.

Software

Programs (set of instructions) and data.

Computer

Desktop Computer (System Unit)

Flat-panel Display (Output Device)

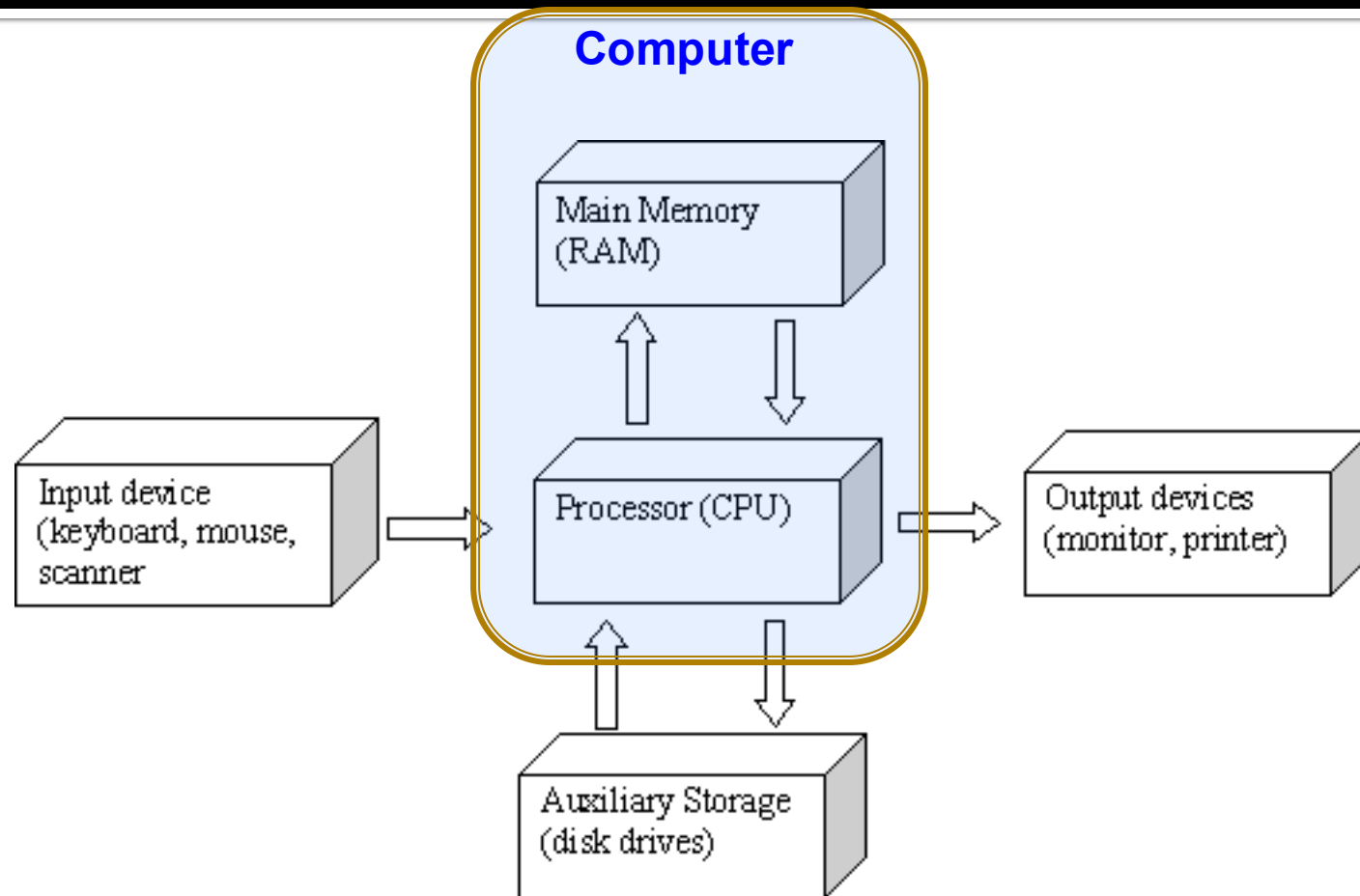
Speaker (Output Device)

Keyboard (Input Device)

Mouse (Input Device)



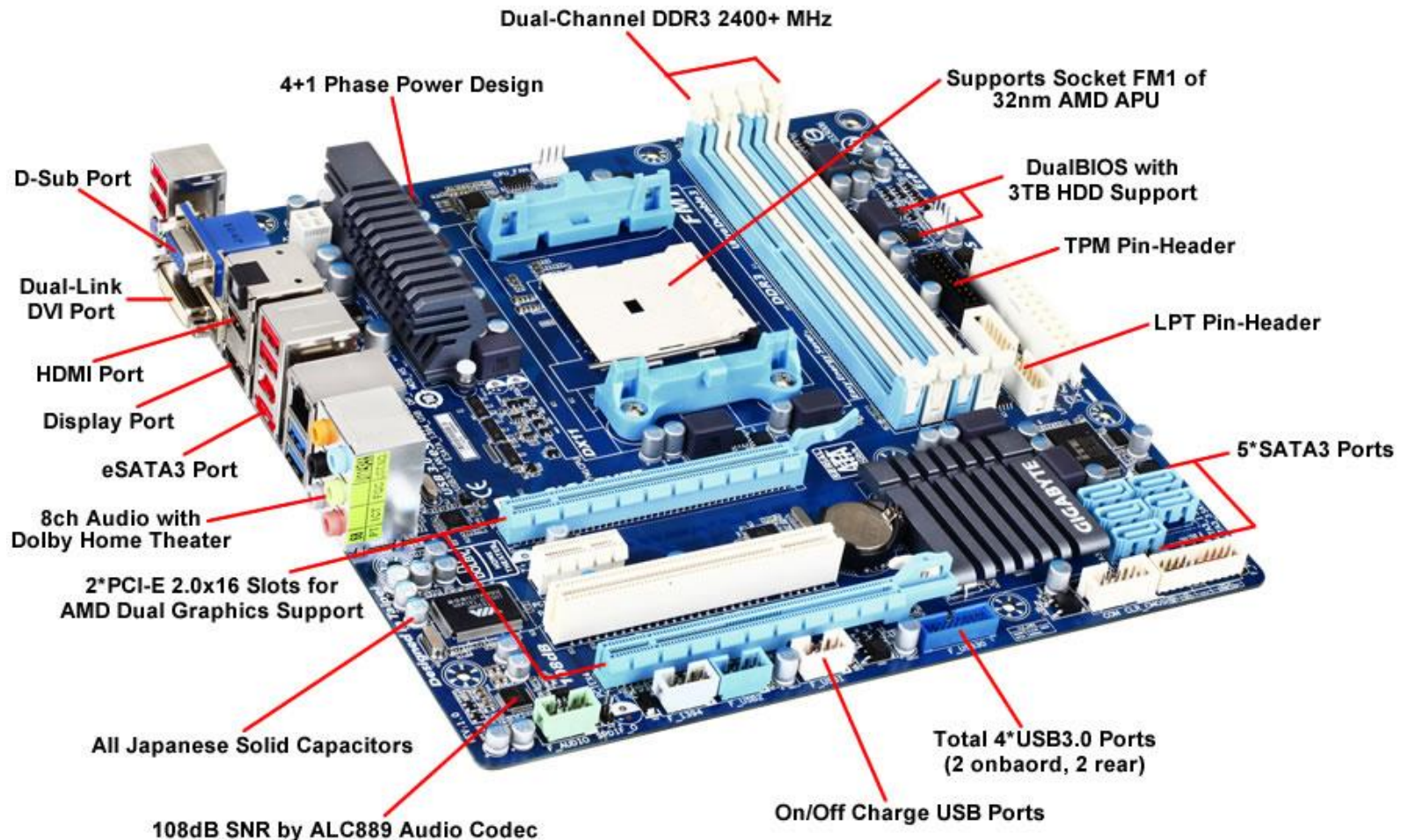
Hardware



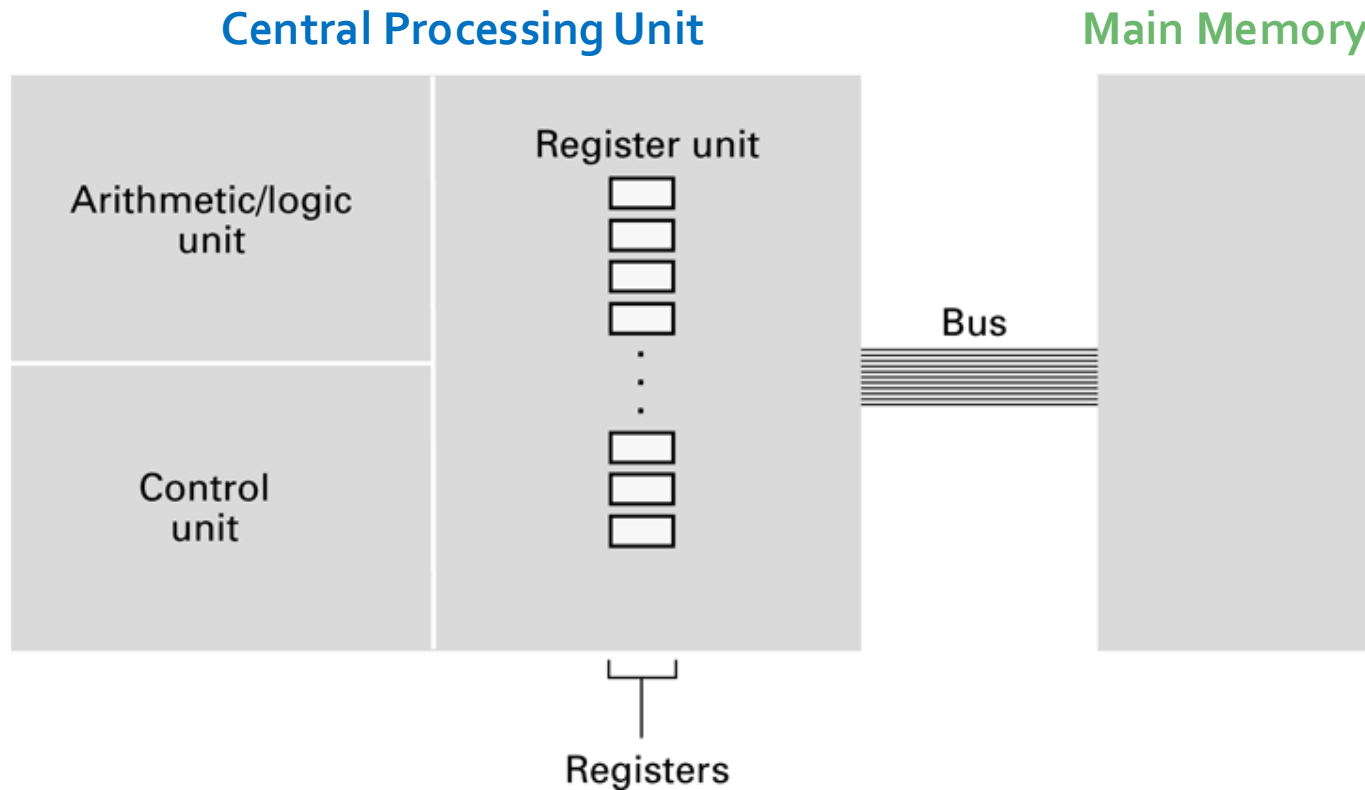
Computing Hardware Essentials

- **Motherboard**
 - Main circuit board of the machine
- **Central Processing Unit (CPU) or processor**
 - primary component responsible for executing instructions and managing tasks.
- **Main Memory (RAM)**
 - High-speed, volatile temporary storage
 - Data/instructions to be used by CPU
- **Bus**
 - Collection of wires used to transfer bit patterns between CPU and Main Memory

Motherboard

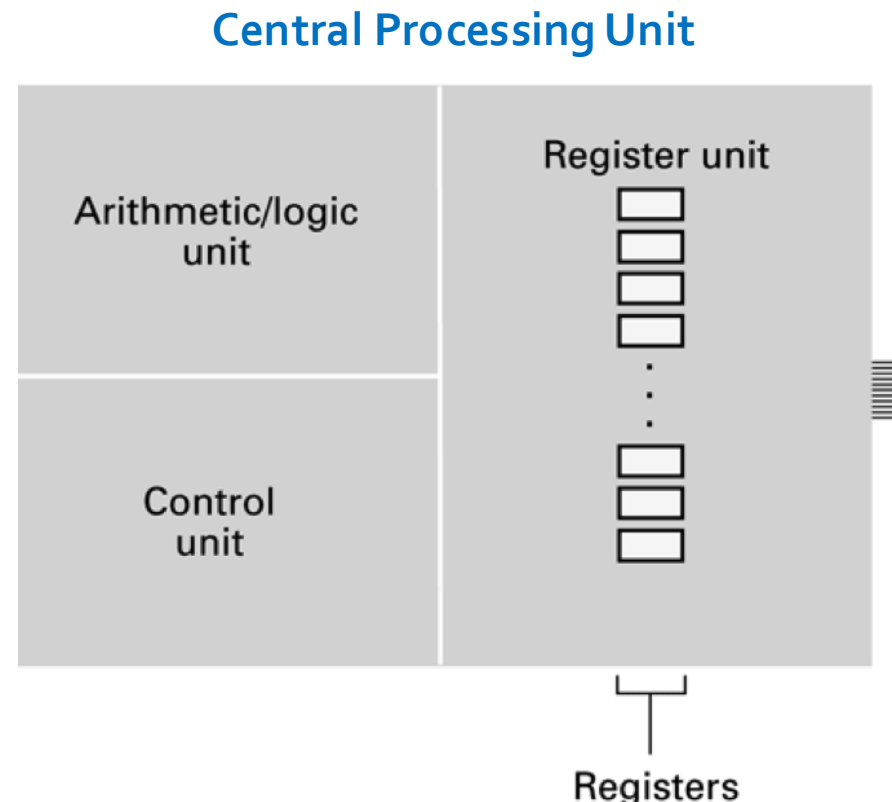


CPU and main memory connected via a bus



Central Processing Unit (CPU)

- **Arithmetic/Logic unit (ALU)**
 - Circuitry that performs arithmetic and logical operations on data
- **Control unit (CU)**
 - Circuitry that direct and coordinate the operations of the processor by controlling the flow of data and the execution of instructions.
- **Registers**
 - General purpose
 - Temporary holding place for data being manipulated by CPU (inputs to ALU, results produced by ALU)
 - Special purpose
 - Particular use – more later



Key Things to Remember

- CPU = ALU + Control Unit
- Control Unit handles program execution.
- ALU handles for simple math operations.

Stored Program Concept

- Crucial to the flexibility of modern computer systems
- Program instructions can be encoded as bit patterns and stored in memory
- CPU extracts instructions and executes them
- The program can be easily altered

Terminology

- **Machine instruction:** An instruction (or command) encoded as a bit pattern recognizable by the CPU.
- **Machine language:** The set of all instructions recognized by a machine. CPUs process their own machine code instruction set. There are 2 common types of computer architectures:
 - RISC
 - CISC

CPU processing: CISC

- **Complex Instruction Set Computing (CISC)**
 - CISC processors architectures have a large and complex set of instructions
 - Fewer instructions are required to perform a task, which leads to more compact machine code.
 - In one instruction they can (1) load value from memory, (2) add it to another value, (3) store the result in memory.
 - CISC processors are more hardware-complex, requiring more circuits to decode and execute instructions
 - Commonly found in desktops and servers: Intelx86, AMD

CPU processing: RISC

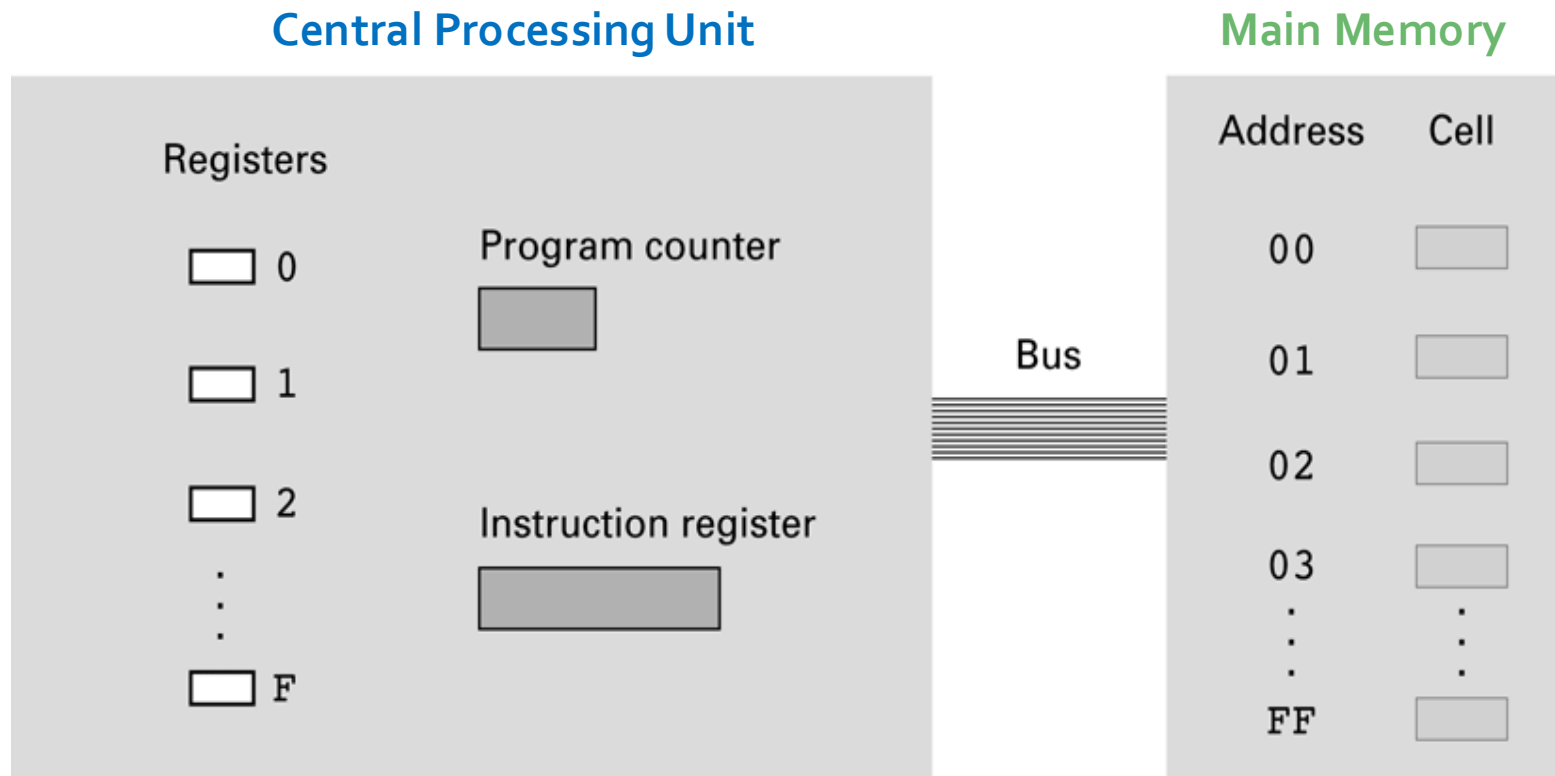
- **Reduced Instruction Set Computing (RISC)**
 - RISC architectures use a **small and simplified set of instructions**.
 - Each instruction is designed to perform a simple task that can typically be executed in a **single clock cycle**.
 - RISC processors are hardware-simpler, with fewer transistors and less consumption of power.
 - RISC architectures are typically used in mobile devices and embedded systems: PowerPC from Apple/IBM/Motorola and ARM (mobile), MIPS (derivative used by PlayStation).

Machine Instruction Types

- **Data Transfer:** copy data from one location to another
- **Arithmetic/Logic:** use existing bit patterns to compute new bit patterns
- **Control:** direct the execution of the program

Simple Machine components

We will use a simple but representative machine to explain how computers work.



Simple Machine components

Machine Architecture

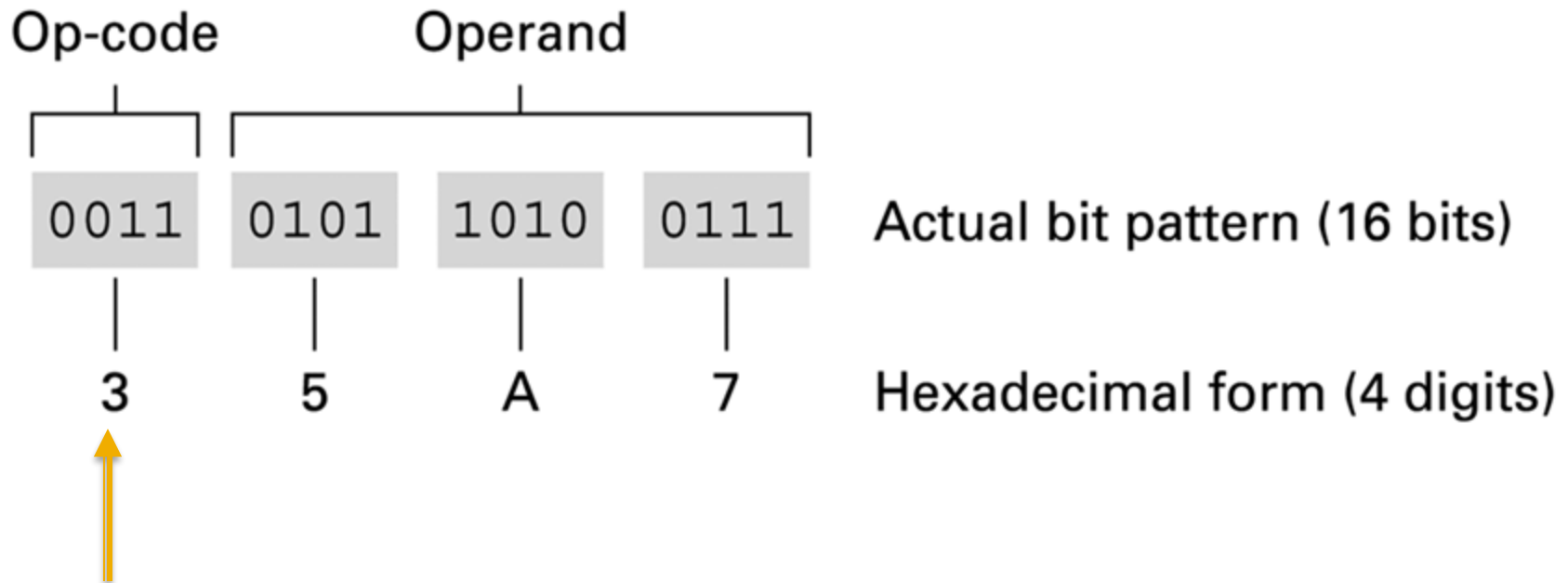
- 16 general-purpose registers numbered 0 through F (in hexadecimal)
 - Each register is one byte (eight bits) long
- 256 cells in the machine's main memory
 - Each cell is assigned a unique address represented by a hexadecimal value in the range of 00 to FF (0 to 255 in decimal)

Parts of a Machine Instruction

Machine's Language

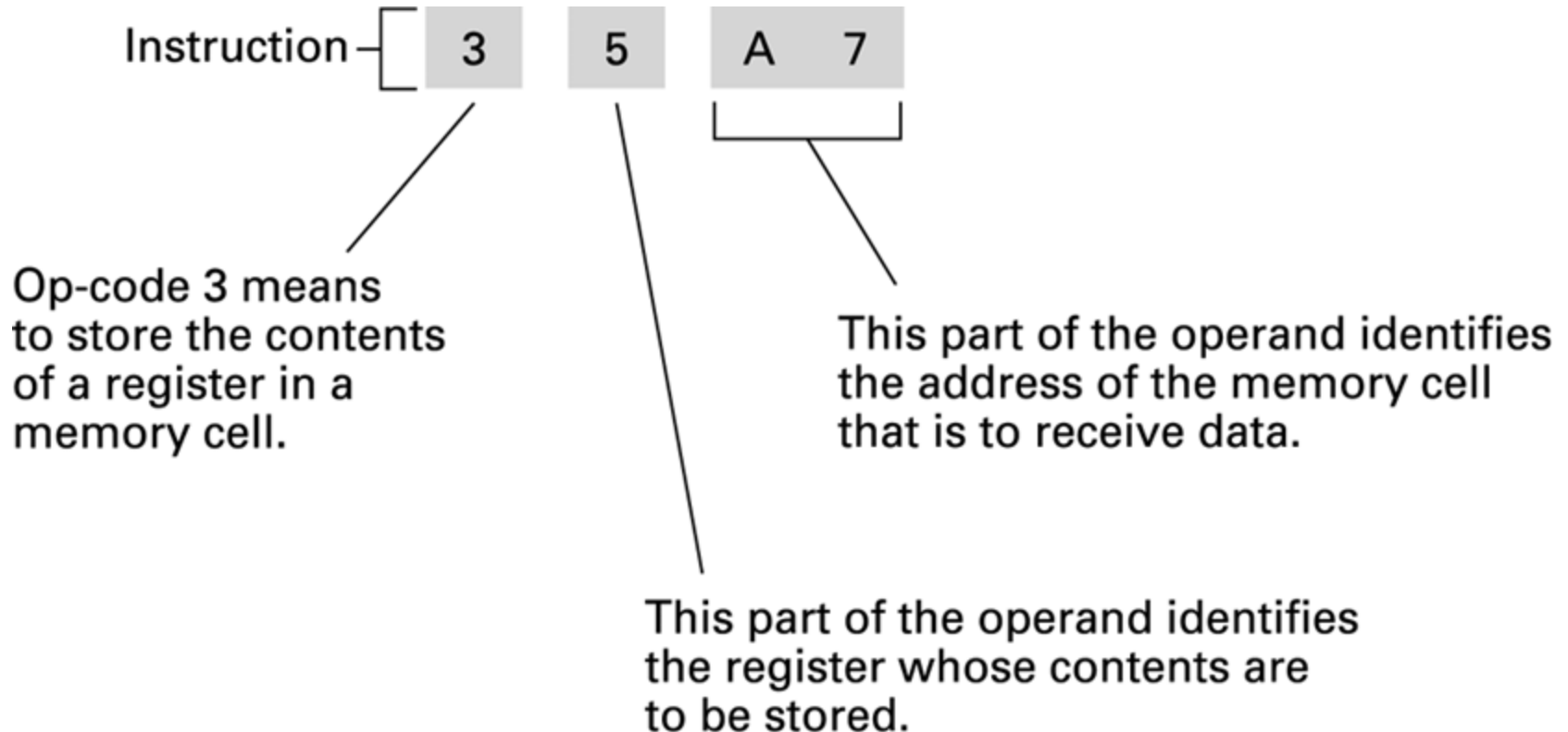
- Each machine instruction is 2 bytes = 16 bits
 - **Op-code (4 bits):** Specifies which operation to execute
 - **Operand (12 bits):** Gives more detailed information about the operation
 - Interpretation of operand varies depending on op-code

The composition of an instruction for our example machine



Examples: MOV, ADD, SUB, JMP, CALL, CMP

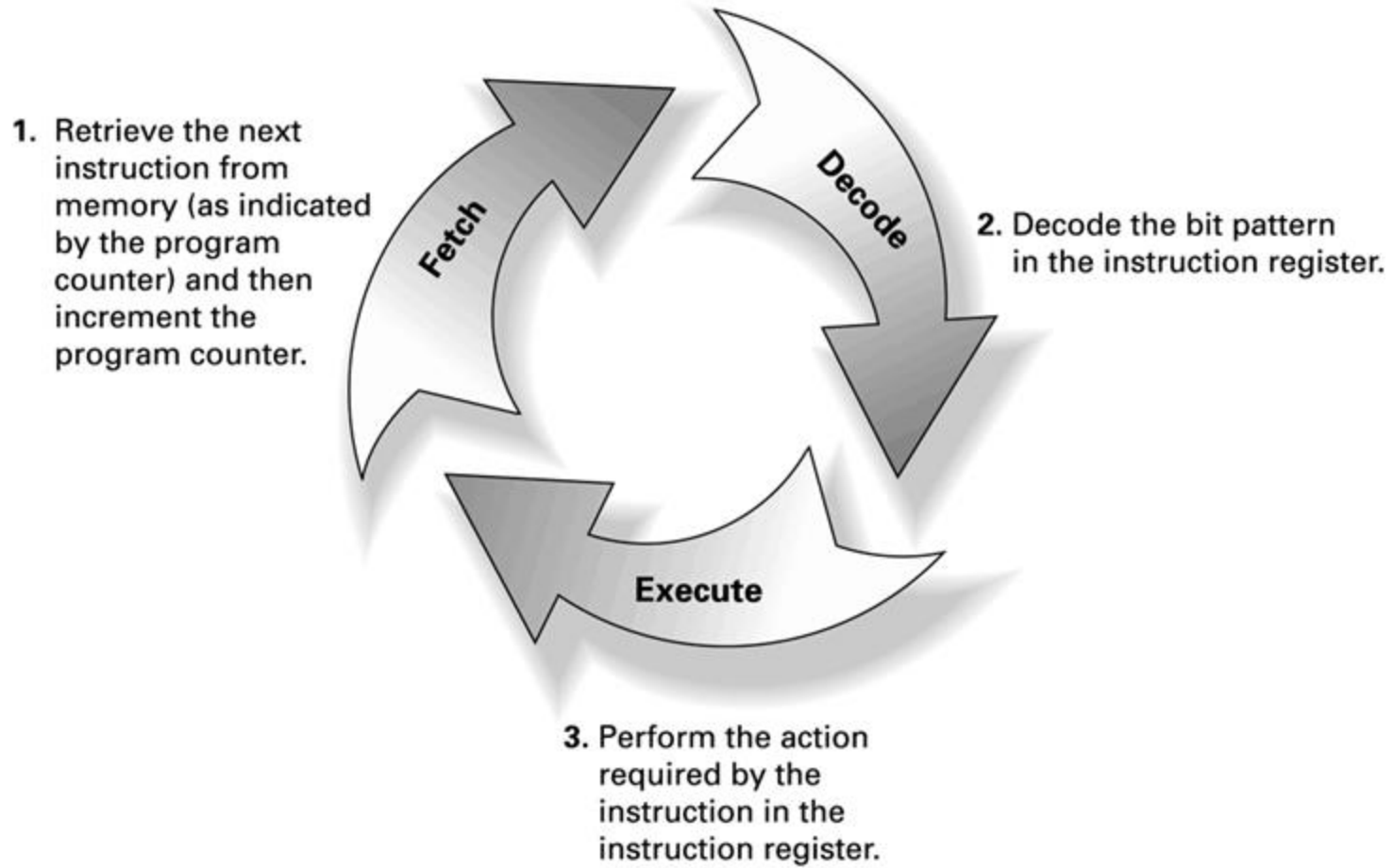
Decoding the instruction 35A7



Program Execution

- Controlled by three special-purpose registers
 - ***Program counter register***: holds **address** of the **next instruction** to be executed in the instruction cycle.
 - ***Instruction register***: holds the **current instruction** being executed from memory
 - ***Accumulator register***: store intermediate results during arithmetic and logic operations.

The Fetch Execute Cycle



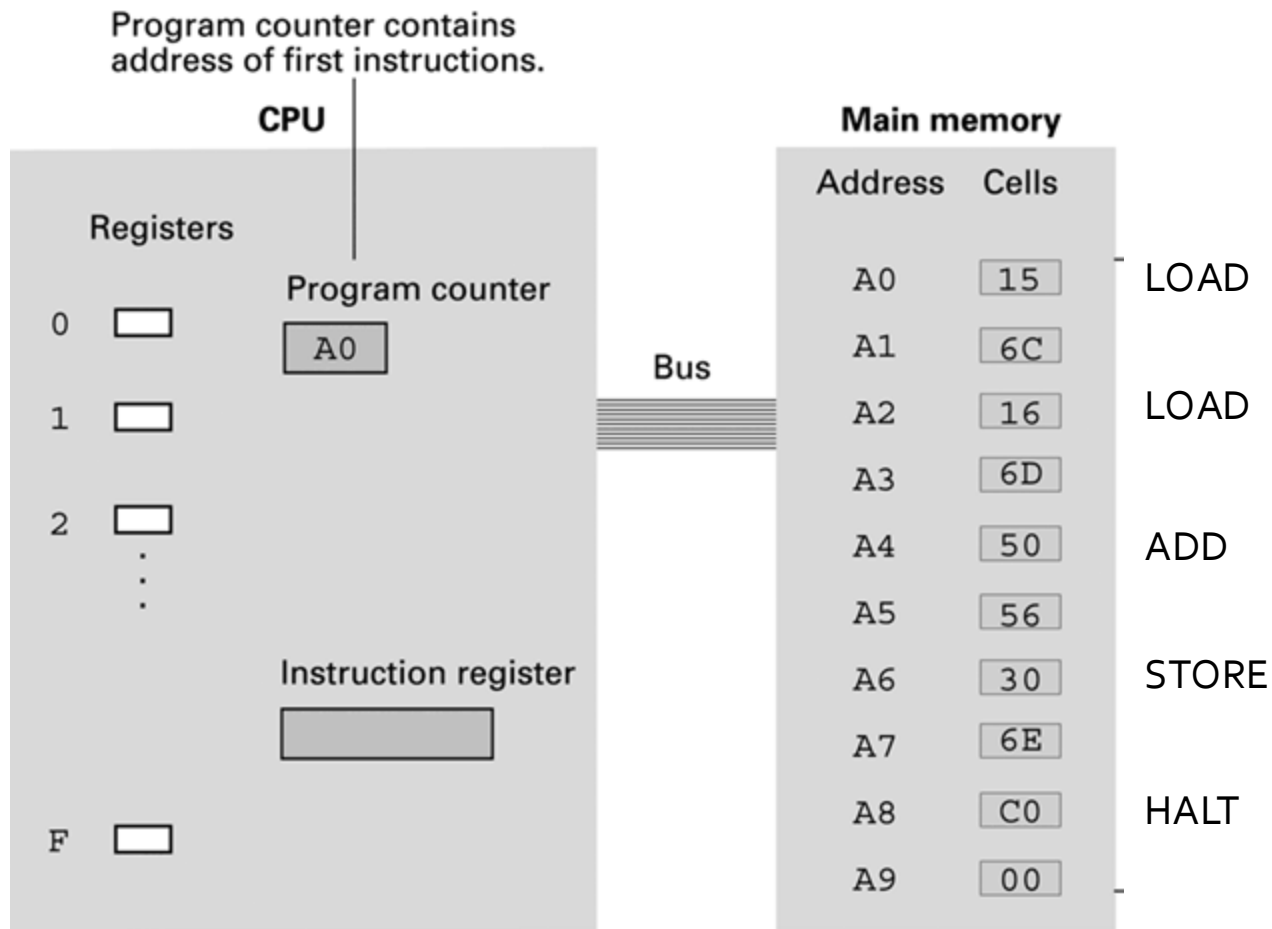
Example: Adding values stored in memory

- Step 1.** Get one of the values to be added from memory and place it in a register.
- Step 2.** Get the other value to be added from memory and place it in another register.
- Step 3.** Activate the addition circuitry with the registers used in Steps 1 and 2 as inputs and another register designated to hold the result.
- Step 4.** Store the result in memory.
- Step 5.** Stop.

An encoded version of some of the machine instructions

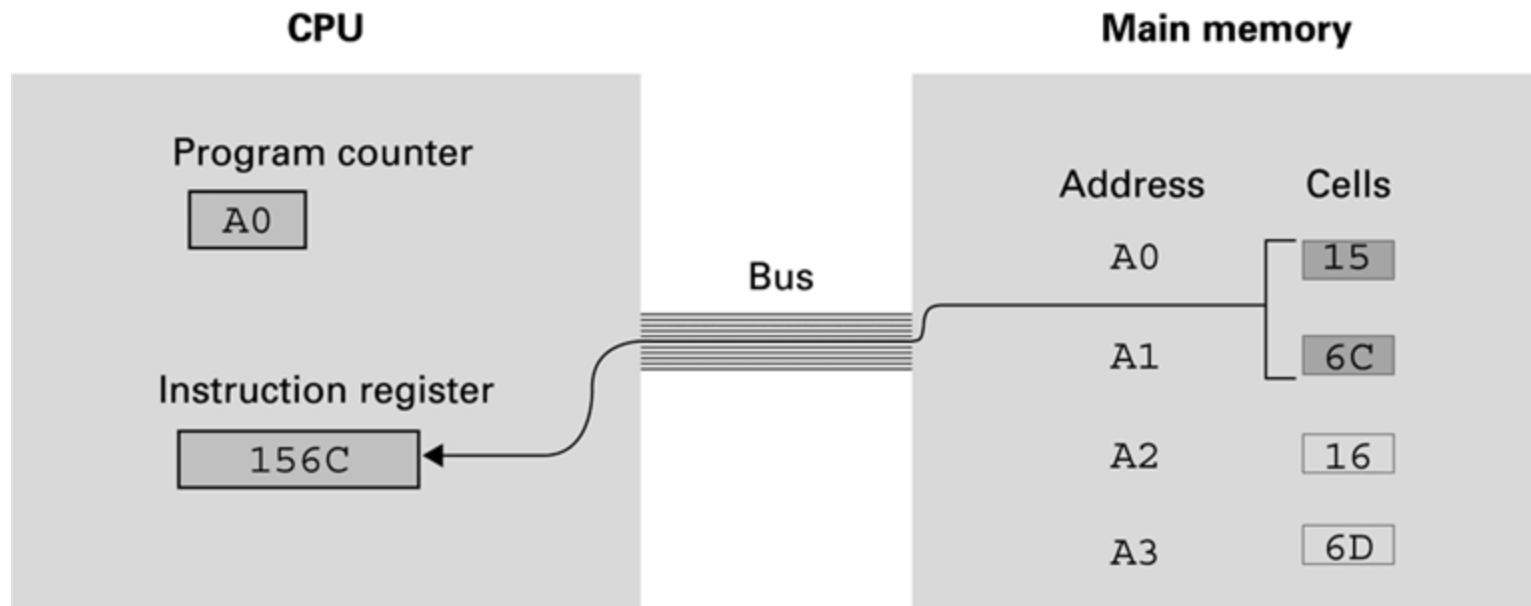
Encoded instructions	Translation
156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
C000	Halt.

The example program from stored in main memory ready for execution



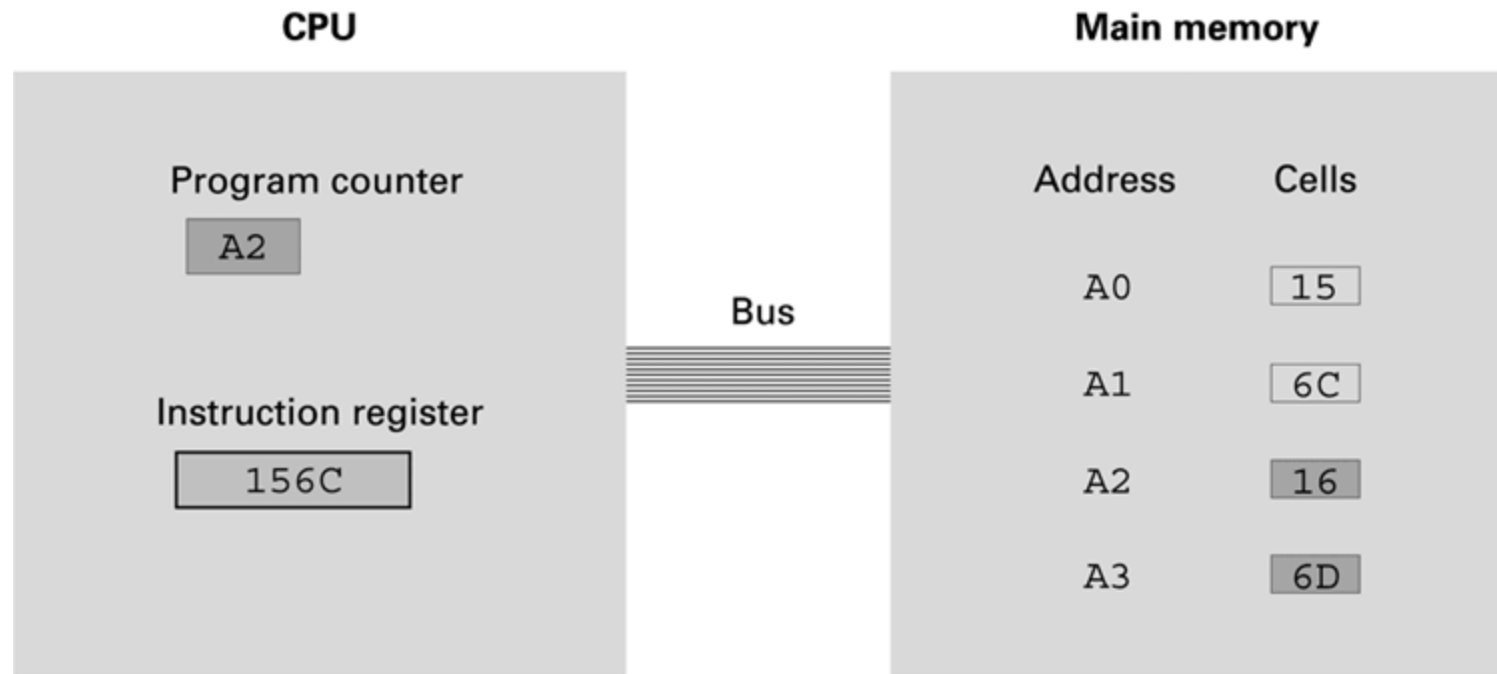
Program is stored in main memory beginning at address A0

Performing the fetch step of the machine cycle



- a. At the beginning of the fetch step the instruction starting at address A0 is retrieved from memory and placed in the instruction register.

Performing the fetch step of the machine cycle (cont'd)

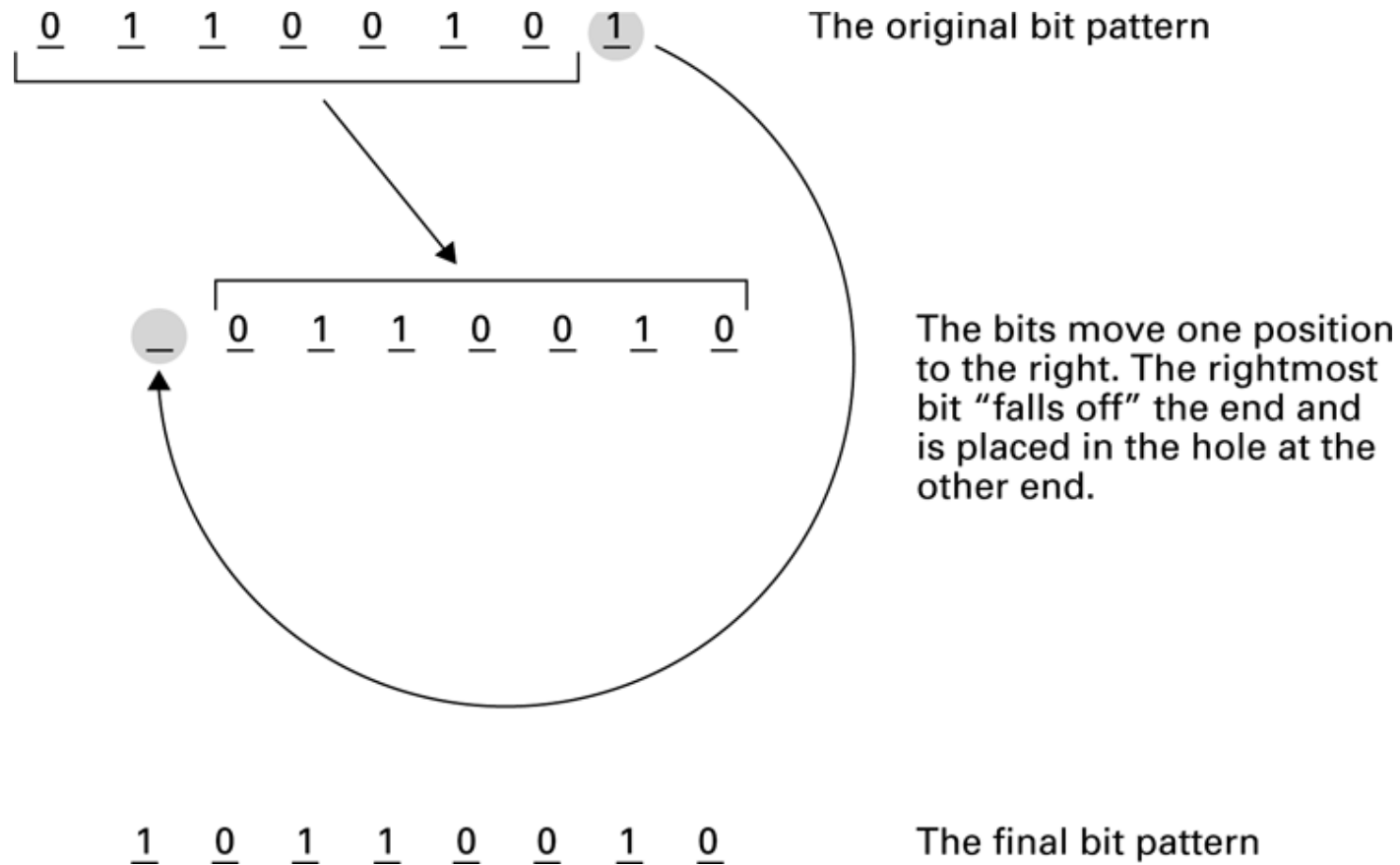


b. Then the program counter is incremented so that it points to the next instruction.

Arithmetic/Logic Operations

- **Logic: AND, OR, XOR**
 - Masking
- **Rotate and Shift:** circular shift, logical shift, arithmetic shift
- **Arithmetic:** add, subtract, multiply, divide
 - Precise action depends on how the values are encoded (two's complement versus floating-point).

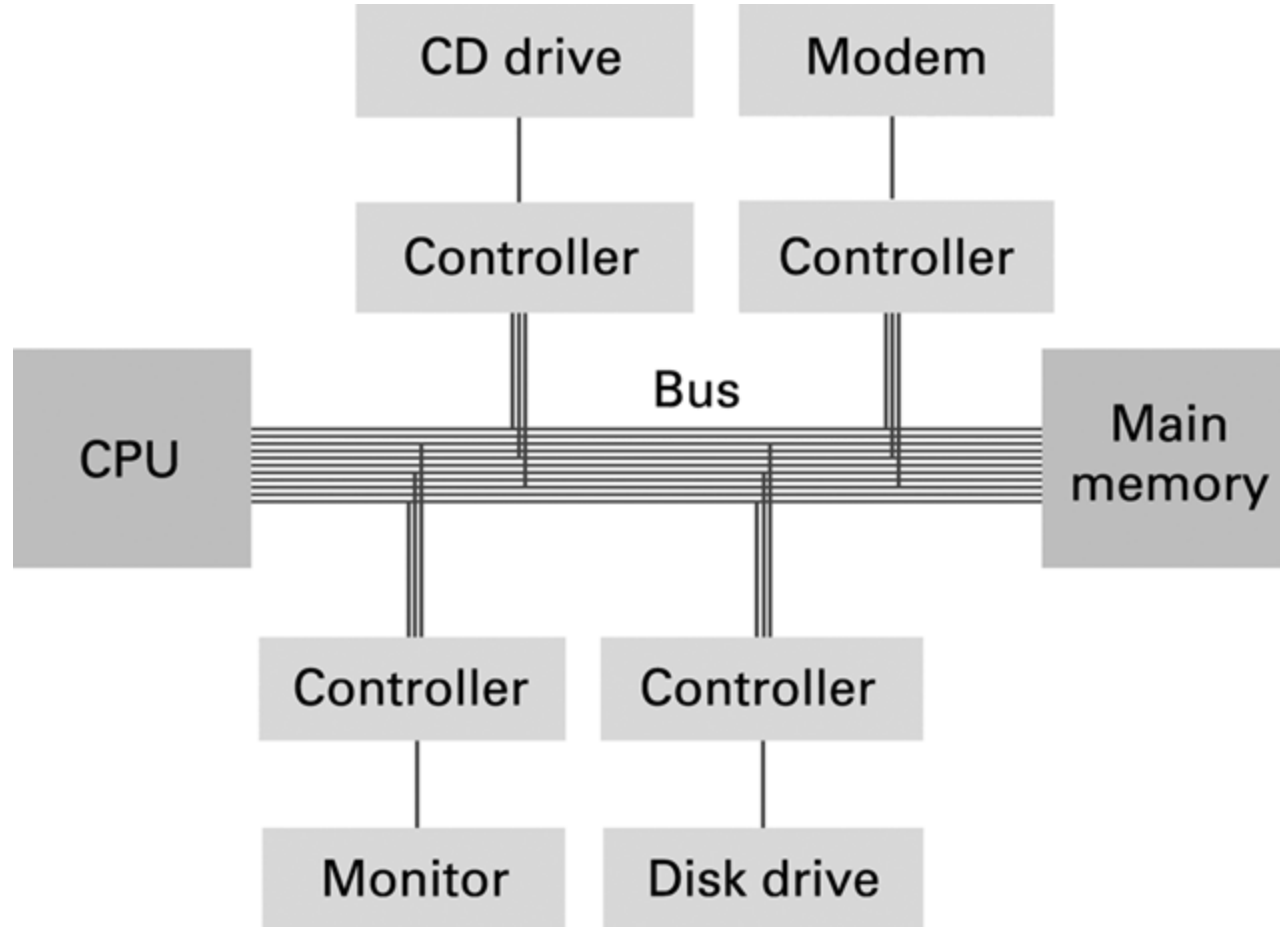
Rotating the bit pattern 65 (hexadecimal) one bit to the right



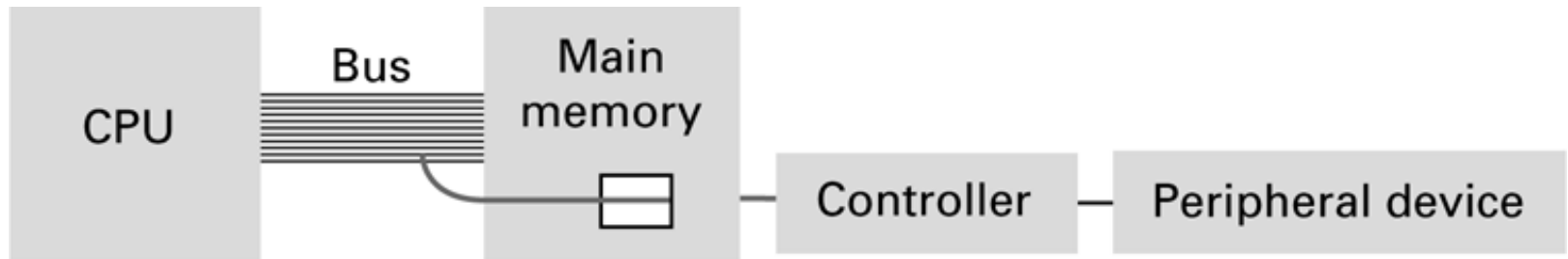
Communicating with Other Devices

- **Controller:** An intermediary apparatus that handles communication between the computer and a device
 - Specialized controllers for each type of device
 - General purpose controllers (USB and FireWire)
- **Port:** The point at which a device connects to a computer
- **Memory-mapped I/O:** CPU communicates with peripheral devices as though they were memory cells. This simplifies communication between the CPU and the peripherals.

Controllers attached to a machine's bus



A conceptual representation of memory-mapped I/O



Examples: GPUs, Network Interface Controlled (NIC), Sound Cards, Keyboard and Mouse Controllers, etc..

Communicating with Other Devices

(continued)

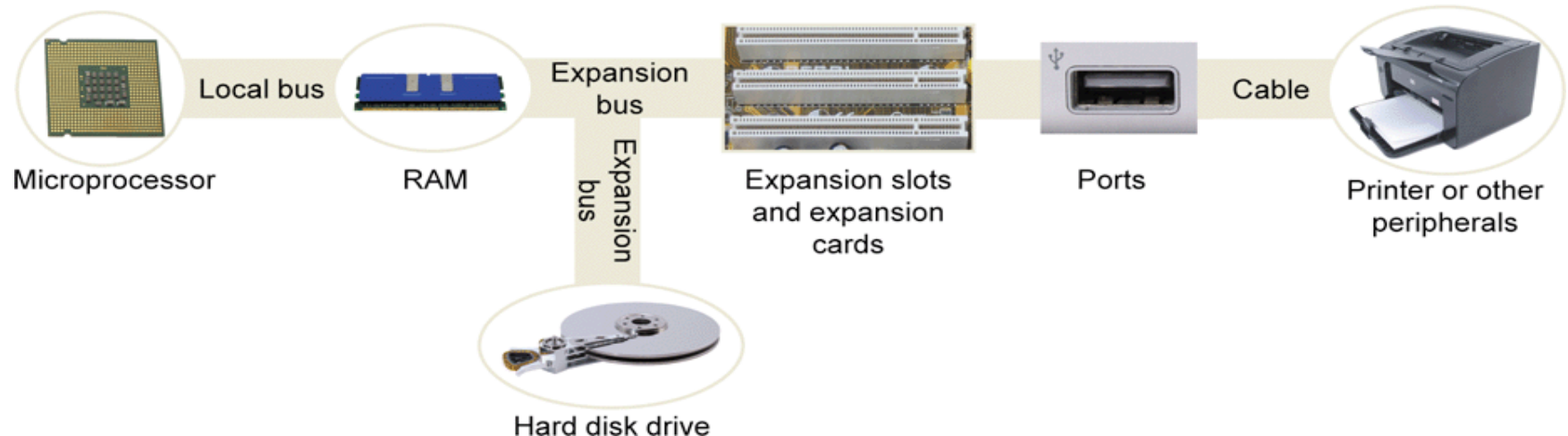
- **Direct memory access (DMA):**
 - Main memory access by a controller over the bus
- **Von Neumann Bottleneck:**
 - Insufficient bus speed impedes performance
- **Handshaking:**
 - The process of coordinating the transfer of data between components

Communicating with Other Devices (continued)

- **Parallel Communication:** Several communication paths transfer bits simultaneously.
- **Serial Communication:** Bits are transferred one after the other over a single communication path.

FIGURE 2-51

The expansion bus connects the computer system board to peripheral devices.



Data Communication Rates

- **Bandwidth:** Maximum available rate at which data can be transmitted over a network or communication channel in a given amount of time.
- **Measurement units**
 - bps: Bits per second
 - Kbps: Kilo-bps (1,000 bps)
 - Mbps: Mega-bps (1,000,000 bps)
 - Gbps: Giga-bps (1,000,000,000 bps)

System Clock

The components of the computer must interact in a synchronized way.

A system clock is used to control the timing of all computer operations.

It generates regular “ticks” (electronic pulses) that set the operating pace of the components.

Its speed is expressed in Megahertz (MHz) or Gigahertz (GHz).